

MoneyPAL Genesis Intelligence Console

Technical Documentation

July 7, 2026
Engineering Team

CONTENTS

1	Executive Summary	3
2	System Architecture	3
2.1	High-Level Data Flow	3
2.2	Full Codebase Architecture Diagram	4
2.3	Infrastructure	6
3	Component Deep Dive	6
3.1	The RAG Engine (packages/genesis_core)	6
3.2	Intelligence Services (backend/app/services)	7
3.2.1	Macro-Economic Intelligence (macro.py)	7
3.2.2	Competitive Intelligence (competitive.py)	7
3.2.3	Regulatory Intelligence (regulatory.py)	7
3.2.4	Policy Formulation (policy.py)	7
3.2.5	Platform Services (platform.py)	8
3.3	Supporting Infrastructure Services	8
3.3.1	Smart Brief Cache (brief_cache.py)	8
3.3.2	Review Queue (review_store.py)	8
3.3.3	Dashboard Feeds (intelligence.py)	8
3.4	Config-Driven Registry (backend/registry)	8
3.5	Authentication & Roles (routes/auth.py)	8
3.6	Frontend (Next.js)	9
4	Key Workflows	10
4.1	The Ask Genesis Query Loop	10
4.2	Document Ingestion (Offline)	10
4.3	Registry Onboarding (Institutions & Regulations)	10
4.4	Policy Brief Loop	10
4.5	Review & Dashboard Loop	11
5	Configuration Reference (.env)	11

1 EXECUTIVE SUMMARY

The **Moneypal Genesis Intelligence Console** is a regulatory, competitive, and macro-economic intelligence platform built for **GICC**, a Karnataka co-operative bank, to support credit assessment and regulatory compliance. It is the first phase of the broader **Moneypal Genesis Layer** onboarding journey and is powered by the **Aroha RAG Framework**. The platform is composed of:

- a **Macro-Economic Intelligence** module with real-time dashboards for national growth indicators, Karnataka state-level indicators, MSME lending trends, and an AI executive briefing
- a **Competitive Intelligence** module with config-driven institution profiles, automated SWOT analyses, and a Karnataka lending-landscape briefing
- a **Regulatory Intelligence** module monitoring RBI master directions, digital lending controls, KYC/AML obligations, and prioritized compliance alerts
- an **Ask Genesis** conversational surface providing grounded, citation-backed natural-language question answering across every indexed collection
- a **Policy Formulation Workspace** that synthesises cross-collection policy briefs for the GICC Policy Maker
- operational surfaces: an **Intelligence Review Queue**, an **Admin / Platform Status** panel, and executive dashboard feeds (**Recently Updated Intelligence** and **Action Items**)

The platform is implemented as a **FastAPI backend** plus a **Next.js frontend**, with a shared **genesis_core** Python package providing the direct (framework-free) RAG engine. **Qdrant** stores dense BAAI/bge-m3 embeddings, **Groq** (llama-3.3-70b-versatile) performs generation, and a **SQLite brief cache** keeps LLM output stable and cheap between page loads.

2 SYSTEM ARCHITECTURE

2.1 High-Level Data Flow

The architecture follows a modular ingestion-processing-serving pipeline in which every intelligence module shares the same retrieval engine, response contract, and cache layer.

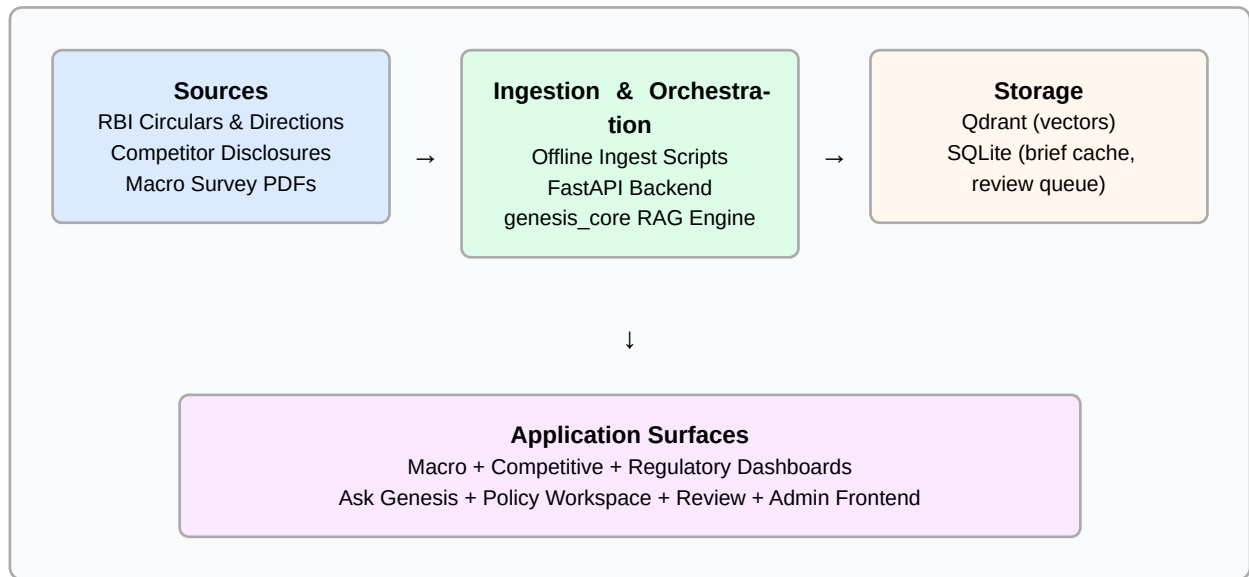
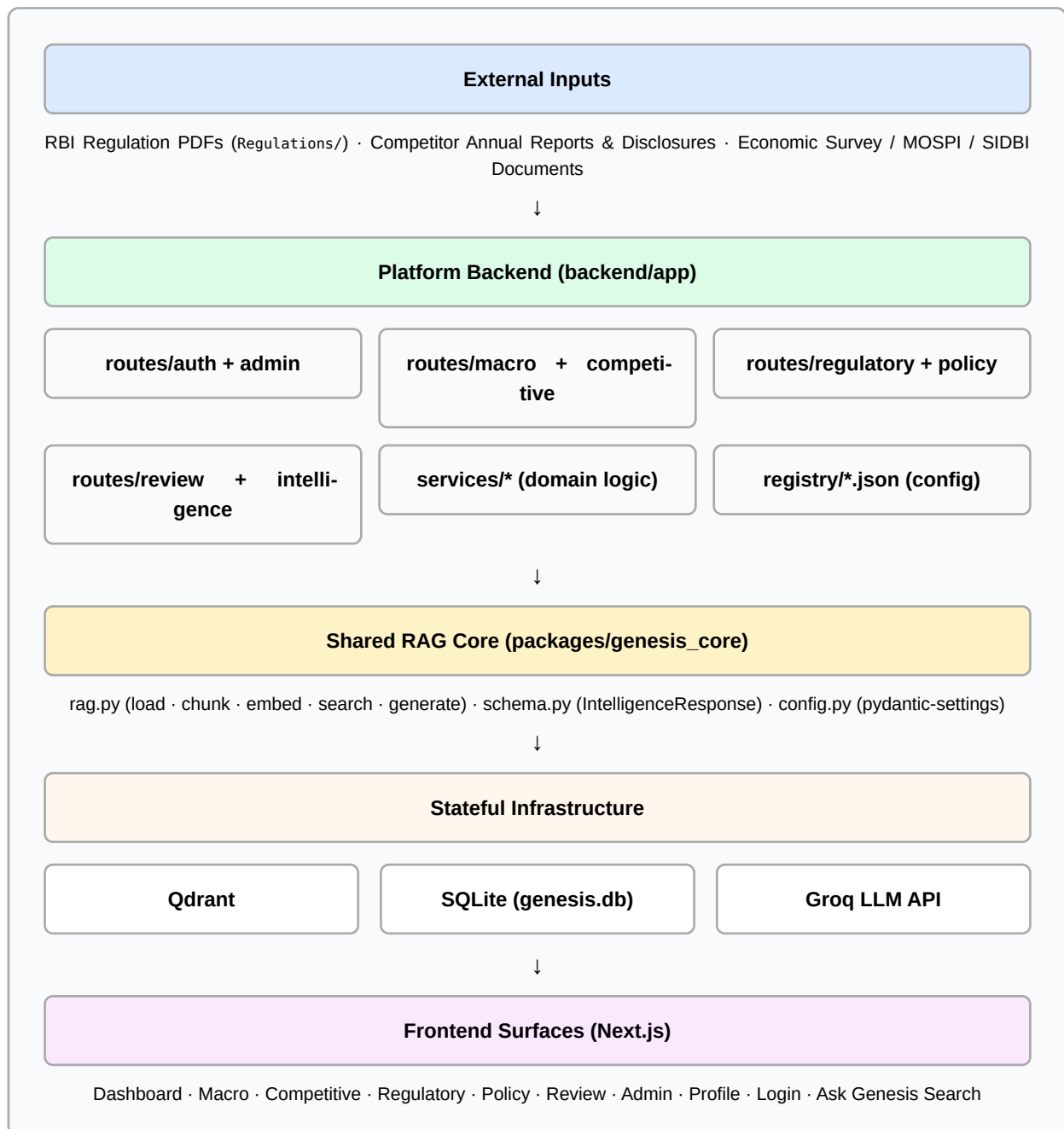


Figure 1: System Data Flow Architecture

2.2 Full Codebase Architecture Diagram

The following diagram captures the major modules and data flows across the entire codebase using native Typst elements so it renders directly in the generated PDF.



Listing 1: Rendered full-codebase architecture diagram

- **Ingestion Layers:**

- **Regulatory Documents:** Official RBI PDFs organized under Regulations/ by category (Digital Lending, KYC/AML, Master Directions, Prudential Norms, etc.) are chunked and embedded into per-category Qdrant collections (reg_*).
- **Competitive Documents:** Public disclosures for each tracked institution are embedded into per-institution collections (comp_*).
- **Macro Documents:** Economic Survey, MOSPI, and SIDBI material feed a single macro intelligence collection.

- **Orchestration:**

- A **FastAPI** application (app/main.py) mounts eight thin routers — auth, macro, competitive, regulatory, admin, review, policy, and intelligence — over shared domain services.

- **Ingestion is offline:** `backend/scripts/ingest.py` runs the load-chunk-embed-upsert pipeline per module before the API serves traffic.
- **Application Modules:**
 - `services/macro.py` generates the four macro briefs (snapshot, Karnataka, MSME, executive briefing) from the macro collection.
 - `services/competitive.py` produces institution profiles, SWOT analyses, and the lending-landscape brief from per-institution collections.
 - `services/regulatory.py` produces director-level regulatory briefings and structured compliance alerts per regulation category.
 - `services/policy.py` synthesises cross-collection policy briefs from any mix of regulations and institutions.
 - `services/platform.py` powers Ask Genesis (cross-collection QA) and hybrid search, plus the admin status endpoint.
 - `services/brief_cache.py`, `services/review_store.py`, and `services/intelligence.py` provide caching, the review queue, and the dashboard feeds respectively.
- **Presentation:**
 - A **Next.js** frontend provides role-aware dashboards, intelligence pages, the policy workspace, the review queue, and platform administration.

2.3 Infrastructure

The system is containerized using Docker Compose behind an Nginx reverse proxy:

- **Backend (backend/Dockerfile):** FastAPI + Uvicorn serving all intelligence APIs; mounts Regulations/ read-only and points registry and index paths at in-container locations.
- **Frontend (node-based build):** Next.js application built with `NEXT_PUBLIC_API_URL=/api` so all browser calls are same-origin.
- **Nginx (nginx:alpine):** Reverse proxy exposed on the host — `/api/*` is prefix-stripped and routed to the backend on port 8000; everything else routes to the frontend on port 3000.
- **Qdrant:** Vector database, either a shared Tailscale/LAN instance (default `192.168.1.183:6333`) or a local container.
- **Groq API:** External LLM inference; a secondary API key automatically takes over when the primary key approaches its rate limit or returns 429.

For non-container development, `backend/scripts/run_backend.sh` sets the dynamic-linking paths (`LD_LIBRARY_PATH`) required on NixOS hosts.

3 COMPONENT DEEP DIVE

3.1 The RAG Engine (packages/genesis_core)

The shared core is a deliberately framework-free RAG engine — five explicit, debuggable steps:

load (pypdf) → **chunk** → **embed** (BAAI/bge-m3) → **store / search** (Qdrant) → **generate** (Groq)

Key properties:

- **Lazy, cached resources:** the embedding model, Qdrant client, and per-query embeddings are memoized (`lru_cache`), so multi-collection retrieval reuses query vectors.
- **Non-destructive collections:** `ensure_collection()` creates a collection only if missing — re-ingestion never wipes existing data.
- **Uniform response contract:** every intelligence endpoint returns an `IntelligenceResponse` (title, AI summary, key points, source reference with document/URL/page, AI note, confidence, last-updated date), defined once in `schema.py`.
- **Central configuration:** `config.py` uses `pydantic-settings` with relative `.env` fallbacks so services launched from any directory pick up the repo-root configuration.

3.2 Intelligence Services (backend/app/services)

3.2.1 Macro-Economic Intelligence (`macro.py`)

Four LLM-generated briefs — **India Economic Snapshot**, **Karnataka Economic Landscape**, **MSME Lending Trends**, and the **AI Executive Briefing** — are produced by grounded generation over the macro collection using curated retrieval query sets from `prompts.py`. Each brief carries fixed key-point scaffolding, a canonical source document/URL, and a confidence rating.

3.2.2 Competitive Intelligence (`competitive.py`)

Tracks eleven Karnataka MSME-lending institutions (co-operative banks, KSFC, SIDBI, Kinara Capital, and others). For each institution it generates:

- **Institution Profile:** who they are, products and pricing, financial strength, and an explicit “threat to GICC” assessment.
- **SWOT Analysis:** structured strengths/weaknesses/opportunities/threats grounded in the institution’s own collection.
- **Lending Landscape:** a cross-institution briefing of the Karnataka competitive environment.

3.2.3 Regulatory Intelligence (`regulatory.py`)

Generates director-level regulatory briefings per RBI category with a fixed five-section structure (Executive Summary, Applicability, Business Impact, Compliance Actions, Effective Date), specifically addressed to NBFCs below Rs. 500 crore. When the Groq API is unavailable, an **extractive fallback summariser** assembles the briefing directly from retrieved chunks so the surface degrades gracefully. Structured **regulatory alerts** with severity levels feed the dashboard’s action items.

3.2.4 Policy Formulation (`policy.py`)

The GICC Policy Maker selects any combination of regulation categories and competitor institutions plus an optional focus statement. The service retrieves the top chunks from **every selected collection** and performs one grounded generation with a fixed brief structure: policy objective, regulatory basis, competitive context, 3–5 recommended policy positions, and implementation actions with owners.

3.2.5 Platform Services (`platform.py`)

Ask Genesis. Natural-language questions are answered across all configured collections (`macro + every comp_* + every reg_*`) with inline document citations, and answers are cached by normalized question text so repeat questions cost zero LLM tokens.

Hybrid Search. Cross-collection semantic search with a lexical special case: directory-style collections (e.g. the RBI NBFC/Bank registry list) score near zero under vector similarity, so they are answered by an optimized keyword/substring matcher instead. Query scaffolding stopwords and ubiquitous company-name suffixes are stripped to isolate distinctive terms before matching.

Platform Status. Reports registry contents, per-collection vector counts, and Qdrant health for the admin panel, alongside the GICC onboarding journey (Institutional Intelligence → Prosper & Tally Migration → NEST Platform).

3.3 Supporting Infrastructure Services

3.3.1 Smart Brief Cache (`brief_cache.py`)

LLM briefs are expensive (5–15 s of Groq time), rate-limited, and non-deterministic — an executive dashboard should not change content on every tab switch. Generated briefs are stored in SQLite (`vector_store/genesis.db`) keyed by a versioned cache key with a 12-hour TTL. Passing `?refresh=1` on any brief route forces regeneration, and a `CACHE_VERSION` bump invalidates all prior briefs when prompt formats change.

3.3.2 Review Queue (`review_store.py`)

The GICC Administrator can mark any generated intelligence item as **pending**, **reviewed**, or **flagged** with a note. State persists in SQLite and flagged items surface as dashboard action items.

3.3.3 Dashboard Feeds (`intelligence.py`)

Two executive-dashboard widgets are derived rather than hardcoded:

- **Recently Updated Intelligence:** driven by brief-cache generation timestamps; regenerating any brief moves it to the top of the feed, with cache keys resolved back to display titles and module links.
- **Action Items:** merges reviewer-flagged intelligence with high-severity regulatory alerts.

3.4 Config-Driven Registry (`backend/registry`)

Institutions and regulation categories are pure metadata — JSON files under `registry/institutions/` and `registry/regulations/`. Each record carries identity fields, source documents/URLs, applicability and priority metadata, and its dedicated Qdrant collection name (`comp_<slug>/reg_<slug>`). Onboarding a new competitor or regulation requires **no code changes**: the `POST /competitive/institutions` and `POST /regulatory/categories` endpoints slugify the name, validate uniqueness, and write the JSON file; ingestion then populates the referenced collection.

3.5 Authentication & Roles (`routes/auth.py`)

Buildathon-scoped mock authentication issues bearer tokens for four demo personas, and the frontend adapts navigation and capabilities per role:

Role	Surface
admin (Moneypal Administrator)	Platform administration, registry management, cross-collection search, platform status
gicc_admin (GICC Administrator)	Intelligence review queue — approve or flag generated briefs
gicc_policy (GICC Policy Maker)	Policy formulation workspace over selected regulations and institutions
gicc_director (GICC Director)	Executive dashboards, briefs, Ask Genesis

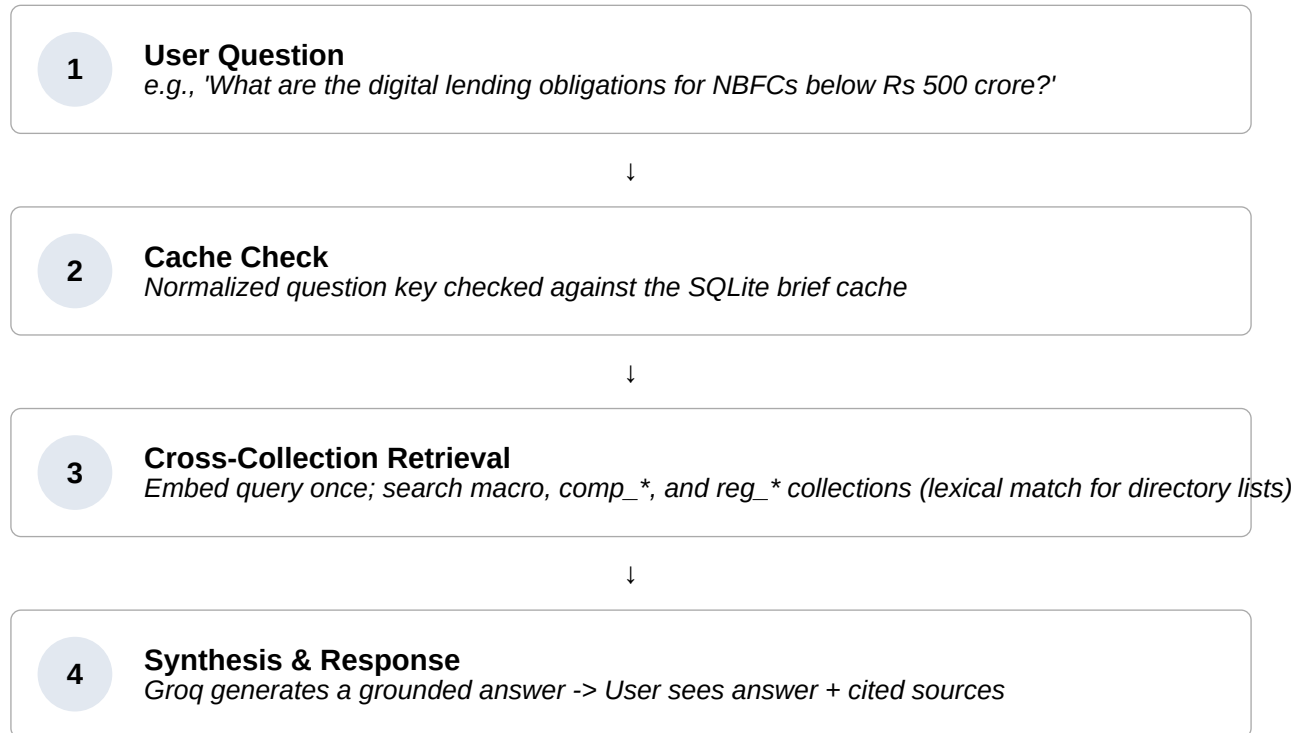
3.6 Frontend (Next.js)

The **Next.js** (TypeScript) frontend is the single operator surface for all modules, styled with Tailwind CSS, Radix UI primitives, and Lucide icons per the design system in DESIGN.md (Pantone 300 U primary blue #005DAA, Harabara display / Inter body pairing).

- **Routes:** dashboard (/), /macro, /competitive, /regulatory, /policy, /review, /admin, /profile, /login.
- **Shared shell:** a persistent app sidebar with role-aware navigation; mobile layouts use a top nav and tab bar, and a PWA registration component supports installability.
- **Intelligence components:** AIBriefPanel, BriefRenderer, IntelligenceCard, SWOTCard, SourceBadge, GenesisSearch, and skeleton/error states render the uniform IntelligenceResponse contract consistently across modules.
- **API client:** a single client in lib/api.ts attaches the bearer token and targets /api/* so Nginx handles routing; lib/useIntel.ts and lib/useUserRole.ts provide data-fetching and role hooks.

4 KEY WORKFLOWS

4.1 The Ask Genesis Query Loop



4.2 Document Ingestion (Offline)

1. **Organize:** PDFs are placed under Regulations/<Category>/ or the competitor data directories.
2. **Ingest:** python backend/scripts/ingest.py regulatory (or competitive) loads each PDF with pypdf, chunks the text, and embeds it with BAAI/bge-m3.
3. **Upsert:** Chunks are written to the category- or institution-specific Qdrant collection with document, page, and URL payload metadata.
4. **Result:** The new material is immediately retrievable by every brief, search, and Ask Genesis query.

4.3 Registry Onboarding (Institutions & Regulations)

1. **Register:** Admin submits a name (plus type/URLs/priority) via the admin panel; the API slugifies it and writes a JSON record under backend/registry/.
2. **Collection:** The record names its dedicated Qdrant collection (comp_<slug> or reg_<slug>).
3. **Ingest:** Source documents for the new entity are ingested into that collection.
4. **Result:** The entity appears across listings, briefs, policy inputs, search, and platform status — with zero code changes.

4.4 Policy Brief Loop

1. **Select:** The Policy Maker chooses regulations, institutions, and an optional focus statement.
2. **Retrieve:** Top chunks are pulled from every selected collection for the focus query.

3. **Synthesize:** One grounded generation produces objective, regulatory basis, competitive context, recommendations, and owned implementation actions.
4. **Cache:** The brief is cached under a composite key of the sorted selections and focus; ? refresh=1 regenerates.

4.5 Review & Dashboard Loop

1. **Generate:** Any brief (re)generation stamps generated_at in the cache, surfacing it in **Recently Updated Intelligence**.
2. **Review:** The GICC Administrator marks items reviewed or flagged with notes in the review queue.
3. **Escalate:** Flagged items and high-severity regulatory alerts merge into the dashboard's **Action Items** widget.

5 CONFIGURATION REFERENCE (.ENV)

Variable	Description	Example
GROQ_API_KEY	Primary Groq LLM key	gsk_...
GROQ_API_KEY_SECONDARY	Failover key (auto-switch on rate limit / 429)	gsk_...
GROQ_MODEL	Reasoning model	llama-3.3-70b-versatile
QDRANT_HOST	Vector DB host	192.168.1.183 or localhost
QDRANT_PORT	Vector DB port	6333
EMBED_MODEL	Embedding model	BAAI/bge-m3
REGULATIONS_DIR	RBI PDF source root	/path/to/ Regulations
REGISTRY_DIR	Regulation registry override (containers)	/srv/backend/ registry/ regulations
LOCAL_INDEX_PATH	Lexical index location (containers)	.../regulatory_chunks.jsonl
NEXT_PUBLIC_API_URL	Frontend API base (build arg)	/api
ALLOW_HASH_EMBEDDINGS	Offline-dev hash-embedding fallback	1 (opt-in only)